1 双目视觉里程计搭建

本次大作业目标是实现一个双目视觉里程计系统，即通过一系列连续的双目图像进行机器人（无人车）的位姿估计（及对应特征点地图的建立）。在实际实现中由于特征点地图比较稀疏不能应用实际，主要是用来对位姿进行校正。, 以下从算法原理、代码实现以及结果分析三方面进行讨论。

2 算法原理

本次作业搭建的 VO 主要由前端和后端两部分组成，前端负责接受每一帧图像进行相机的初始位姿估计，并按一定规则提取关键帧供后端进行优化；后端则是通过接收前端提取出来的关键帧相应位置的位姿和路标点位置进行优化。

2.1 位姿估计

每一时刻双目相机会拍摄左右两张照片，每两张照片为一帧，根据每帧图像进行相机位姿估计的方法是光流：通过提取每帧图像的关键点并观察前后帧的关键点的光流变化进行位姿的估计。位姿估计涉及的算法主要包括：图像关键点提取、通过双目图像关键点进行三角化建立特征点的世界坐标、光流法进行位姿估计，下面一一进行描述。

1. 角点提取

   目前 opencv 库中主流的提取图像角点的方法主要有 SIFT、SURF、ORB、Harris 等方法，考虑到特征点提取速度、区分度等因素，本次作业选用了 opencv 中的 FAST、ORB 和 GFTT 三种类型特征点，下面一一对其原理进行描述。

   SIFT 角点：

   SIFT 角点的基本原理是通过高斯滤波对原图像构建不同尺度的金字塔模型，通过不同尺度间的图像间的像素点灰度值找出在不同尺度下均具有极值的像素点，所以 SIFT 角点最大特点是具有尺度不变性；

   SURF 角点：

   SURF 角点是对 SIFT 角点的进一步优化，通过改变图像金字塔的构造方法避免了降采样，加快了计算速度同时提高了算法鲁棒性；

ORB 角点:，而 FAST 角点的判断方法很简单：通过判断计算图像中每隔像素点周围 16 个角点围成的圈，如果圈中有连续 N 个点的像素值与关注点的灰度值有一定差值，则将其判断为 ORB 角点；

GFTT 角点：

GFTT(Good Feature To Track) 算法主要提取的是 Shi-Tomasi 角点，而 Shi-Tomasi 角点是基于 Harris 的改良版本，Harris 角点提取的基本思想是使用一个固定窗口在图像上进行任意方向上的滑动，比较滑动前与滑动后两种情况，窗口中的像素灰度变化程度，如果存在任意方向上的滑动，都有着较大灰度变化，那么我们可以认为该窗口中存在角点；Shi-Tomasi 角点和 Harris 角点的主要差异是表征窗口滑动前后像素变化的方法不同；

不同角点在效果和性能上的区别如下图所示：

	Corner Detector	Blob Detector	Rotation Invariant	Scale Invariant	Affine Invariant	Repeatability	Localization Accuracy	Robustness	Efficiency
Haris	x		x			+++	+++	++	++
Shi-Tomasi	x		x			+++	+++	++	++
FAST	x		x	x		++	++	++	++++
SIFT		x	x	x	x	+++	++	+++	+
SURF		x	x	x	x	+++	++	++	++
CENSURE		x	x	x	x	+++	++	+++	+++

图 1: 不同角点的比较

2. 左右帧图像角点匹配

   在提取出左图像的角点后，需要找到其在有图像中对应的像素点用于进行三角化，为了减少时间提高系统处理速度，可以直接通过光流计算，具体的计算法在后一部分统一说明。

3. 关键点三角化

在提取了左右图中对应特征点后需要通过三角化恢复其在世界坐标的位置，本次大作业中使用 DLT 方法（direct linear transformation），主要原理如下：

三角化：

设关注点在世界坐标下坐标为 $p_w = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$，相机位姿分别为 $[R_1|t_1] = T_1$，右相机位姿为 $[R_2|t_2] = T_2$，

另外点 p 在两相机的归一化坐标分别是 $p_1 = \begin{bmatrix} X_1/Z_1 \\ Y_1/Z_1 \\ 1 \end{bmatrix}$ $p_2 = \begin{bmatrix} X_2/Z_2 \\ Y_2/Z_2 \\ 1 \end{bmatrix}$

则有 $\Rightarrow \begin{cases} Z_1 p_1 = T_1 p_w \\ Z_2 p_2 = T_2 p_w \end{cases} \xrightarrow{t \times t = 0} \begin{cases} p_1 \times Z_1 p_1 = p_1 \times T_1 p_w = 0 \\ p_2 \times Z_2 p_2 = p_2 \times T_2 p_w = 0 \end{cases}$

$$\Rightarrow \begin{cases} \begin{bmatrix} p_1 \times T_1 \\ p_2 \times T_2 \end{bmatrix} p_w = 0 \end{cases}$$

令 $\begin{bmatrix} p_1 \times T_1 \\ p_2 \times T_2 \end{bmatrix} = A$ $(A \in \mathbb{R}^{6\times4})$，则通过解 $Ap_w = 0$ 可以获得该点的世界坐标。

由于 $A \in \mathbb{R}^{6\times4}$，虽然这是超定方程，$\Rightarrow$ 求解 $\min\|Ap_w\|$ $(p_w \neq 0^{4\times1})$

假设 $\|p_w\| = 1$，通过 SVD 分解 $\Rightarrow A = U\Sigma V^T \Rightarrow \min\|Ap_w\| = \min\|U\Sigma V^T x\|$

$= \min\|\Sigma V^T x\|$

令 $y = V^T x$ $= \min\|\Sigma y\|$

$\because$ Σ 为奇异值矩阵 $\begin{bmatrix} \sigma_1 & & & 0 \\ & \sigma_2 & & \\ & & \sigma_3 & \\ 0 & & & \sigma_4 \\ & & & 0 \end{bmatrix}$ 且 $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \sigma_4$，$\|y\| = 1$

显然当 $y = \begin{bmatrix} 0 \\ \vdots \\ 1 \end{bmatrix}$ 时 $\|\Sigma y\|$ 最小 $\Rightarrow x = Vy$ 即 x 为 V 的最后一列

此时 $\|x\| = 1$，$x = \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \Rightarrow p_w = \begin{bmatrix} X/w \\ Y/w \\ Z/w \\ 1 \end{bmatrix} = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$

至此，三角化完成

图 2: 三角化过程推导

4. 光流

通过追踪前后两帧的路标点的光流变化可以估计出当前相机的初始位姿估计，跟踪路标点变化可以通过上述的特征点提取及匹配完成，具体计算我们通过前后两帧的左右两张图像的两两组合有四种光流计算组合，由于在实现我将左侧图像作为主要考虑图像，因此计算光流时计算的是上一帧的左侧图像和当前帧的左侧图像。LK 光流计算原理如下：

LK 光流的计算基于三个假设：视野亮度保持恒定，视野只进行短距离移动，物体具有空间一致性；

亮度恒定表示视野中像素点在经过一定时间 t 的变化后仍然保持相同的灰度值（亮度值）即：

$$I(x, y, t-1) = I(x + u(x, y), y + v(x, y), t)$$

空间一致性表示参考的像素点周围一定范围的像素点在移动的时候都保持相同的移动距离，假设取窗口大小为 5，对上世进行泰勒级数展开，则考虑 25 个像素点在移动距离为 (u,v) 时满足以下等式：

$$I_t(p_i) + \nabla I(p_i)[u\ v] = 0$$

求解过程如下所示：

$I_t(p_i) + \nabla I(p_i)[u \cdot v]$ $\quad p_i = (x_i, y_i) \quad i \in [1, 25]$

$$\Rightarrow \begin{bmatrix} I_x(p_1) & I_y(p_1) \\ \vdots & \\ I_x(p_{25}) & I_y(p_{25}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(p_1) \\ \vdots \\ I_t(p_{25}) \end{bmatrix} \Rightarrow Ax = b$$

可这个是超定方程，可以求出最小二乘解 $\Rightarrow x = (A^TA)^{-1}A^Tb$

令 $M = A^TA = \begin{bmatrix} \Sigma I_xI_x & \Sigma I_xI_y \\ \Sigma I_xI_y & \Sigma I_yI_y \end{bmatrix}$ ~~为了保证 A^TA 可逆~~

令 λ_1, λ_2 为 M 的两个特征值，为了保证 A^TA 可逆，λ_1, λ_2 应尽可能大，因此只能~~直接~~通过角点进行光流计算，效果才比较好。

图 3: 光流计算过程

同时，为了提高准确性，通常采用构建多层图像金字塔的过程中，先对图像进行下采样在较小尺度上进行光流计算位姿初值，在提高图像尺度进行迭代，能够减少迭代时间及提高稳定性；

5. 位姿估计

   目前已有特征点的世界坐标以及通过光流估计出的特征点在当前帧的图像坐标，可以通过最小化特征点在图像上的重投影误差来进行位姿估计，具体计算过程和后端优化过程比较类似，在位姿优化部分统一说明。

2.2 位姿优化

虽然前端每一帧可以通过光流可以进行相机位姿的估计，但是每一帧都会有一定的漂移，这种误差经过数帧积累后会逐渐增大，所以需要通过后端每隔一段时间进行校正，校正的具体过程是通过前端从所有帧图像中按某种策略选取关键帧并对所有关键帧通过 bundle adjustment 通过最小化重投影误差从而进行累积误差的消除，具体实现原理如下图所示：

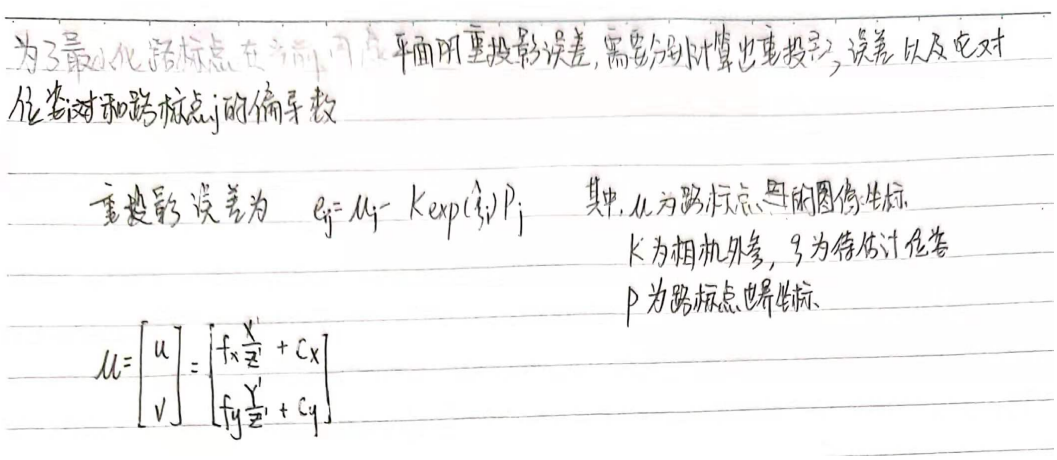
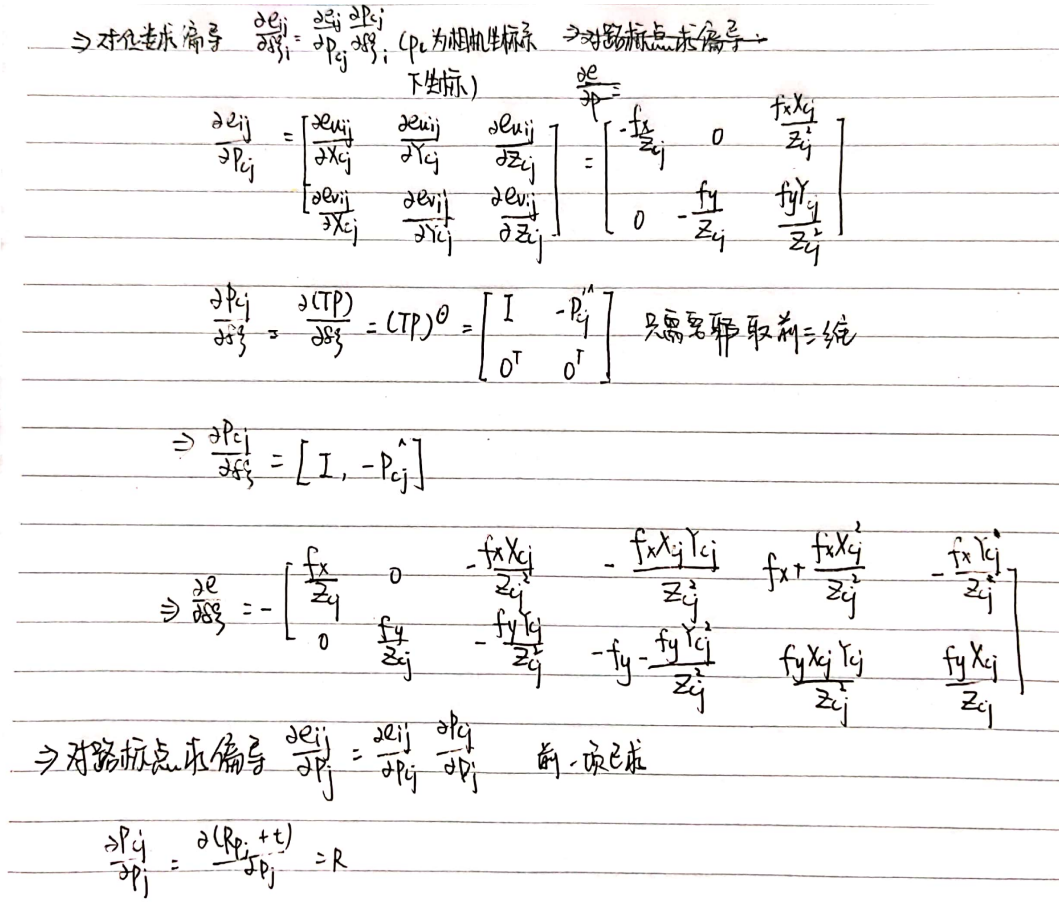


图 4: 位姿优化部分雅克比矩阵推导过程



$\Rightarrow$ 对位姿求偏导 $\frac{\partial e_{ij}}{\partial \xi_i} = \frac{\partial e_{ij}}{\partial P_{cj}} \frac{\partial P_{cj}}{\partial \xi_i}$ (p_c 为相机坐标系下坐标) $\Rightarrow$ 对路标点求偏导

$$\frac{\partial e}{\partial p} = \frac{\partial e_{ij}}{\partial P_{cj}} = \begin{bmatrix} \frac{\partial e_{uij}}{\partial X_{cj}} & \frac{\partial e_{uij}}{\partial Y_{cj}} & \frac{\partial e_{uij}}{\partial Z_{cj}} \\ \frac{\partial e_{vij}}{\partial X_{cj}} & \frac{\partial e_{vij}}{\partial Y_{cj}} & \frac{\partial e_{vij}}{\partial Z_{cj}} \end{bmatrix} = \begin{bmatrix} -\frac{f_x}{Z_{cj}} & 0 & \frac{f_x X_{cj}}{Z_{cj}^2} \\ 0 & -\frac{f_y}{Z_{cj}} & \frac{f_y Y_{cj}}{Z_{cj}^2} \end{bmatrix}$$

$$\frac{\partial P_{cj}}{\partial \delta\xi} = \frac{\partial (TP)}{\partial \delta\xi} = (TP)^{\odot} = \begin{bmatrix} I & -P_{cj}^{\wedge} \\ 0^T & 0^T \end{bmatrix}$$ 只需要取前三维

$$\Rightarrow \frac{\partial P_{cj}}{\partial \delta\xi} = [I, -P_{cj}^{\wedge}]$$

$$\Rightarrow \frac{\partial e}{\partial \delta\xi} = -\begin{bmatrix} \frac{f_x}{Z_{cj}} & 0 & -\frac{f_x X_{cj}}{Z_{cj}^2} & -\frac{f_x X_{cj} Y_{cj}}{Z_{cj}^2} & f_x + \frac{f_x X_{cj}^2}{Z_{cj}^2} & -\frac{f_x Y_{cj}}{Z_{cj}} \\ 0 & \frac{f_y}{Z_{cj}} & -\frac{f_y Y_{cj}}{Z_{cj}^2} & -f_y - \frac{f_y Y_{cj}^2}{Z_{cj}^2} & \frac{f_y X_{cj} Y_{cj}}{Z_{cj}^2} & \frac{f_y X_{cj}}{Z_{cj}} \end{bmatrix}$$

$\Rightarrow$ 对路标点求偏导 $\frac{\partial e_{ij}}{\partial P_j} = \frac{\partial e_{ij}}{\partial P_{cj}} \frac{\partial P_{cj}}{\partial P_j}$ 前一次已求

$$\frac{\partial P_{cj}}{\partial P_j} = \frac{\partial (RP_j + t)}{\partial P_j} = R$$

图 5: 位姿优化部分雅克比矩阵推导过程（续）

3 代码结构

代码框架主要参考十四讲中实践部分代码并进行一定简化，减少了需要的依赖库，整合了部分源代码以及减少配置文件的设置并增加了结果测试模块，下面主要按基本数据结构、前端和后端以及周边配置相关说明。

3.1 基本数据结构

在每次计算位姿中，我们主要需要对帧、特征点已经相应的路标点进行计算，所以基本的数据结构应该包括这三个类，下面对三个类需要的变量进行说明。

1. 帧 (Frame): 主要包括该帧对应的序列号（时间戳）id，对应的左右图像 (left_img, right_img)，相机位姿 (pose_) 以及关联的特征点，由于一帧对应多个特征点，采用两个 vector(left_features_, right_features_) 分别储存其关联的特征点指针；除此之外，我们还需要一个 bool 变量（is_keyframe_）判断它是否为关键帧；

2. 路标点 (MapPoint): 主要信息为其在 map 中的世界坐标系下的位置 (pos_)，同时还需要记录它被观测过的次数 (observed_times) 以及对应的观测值 (observations_)（对应特征点）；

3. 特征点 (Feature)：主要信息为其关联的图像（帧）(frame_) 和路标点 (mappoint_) 以及对应的像素位置 (position_)，同时我们需要两个 bool 量去判断该特征点是否位于左图（主要关注的图）(is_on_left_image_) 以及判断光流追踪是否成功 (is_outlier_)。

4. 说明：由于帧、路标点以及特征点之间存在互相关联，如果都要采用智能指针 shared_ptr 析构时会出现问题，所以统一将所有权放在帧上 (Frame)，对应路标点（MapPoint）以及特征点 (Feature) 中对其他两个类的指针采用弱指针 (weak_ptr)，两者的区别主要在于 shared_ptr 允许多个该智能指针共享“拥有”同一堆分配对象的内存，这通过引用计数（reference counting）实现，会记录有多少个 shared_ptr 共同指向一个对象，一旦最后一个这样的指针被销毁，也就是一旦某个对象的引用计数变为 0，这个对象会被自动删除；相对而言，weak_ptr 是为配合 shared_ptr 而引入的一种智能指针来协助 shared_ptr 工作，它可以从一个 shared_ptr 或另一个 weak_ptr 对象构造，它的构造和析构不会引起引用计数的增加或减少。没有重载 * 和 -> 但可以使用 lock 获得一个可用的 shared_ptr 对象。

3.2 配置类

除了三个主要的数据结构类以外，我们还需要一些配置类来对配置输入以及可视化进行辅助，主要包括系统配置，相机结构，可视化工具以及前后端需要用的 g2o 类型，下面一一进行说明：

1. 系统配置 config：这一部分主要 include 了所有系统需要的库以及声明一些需要的类以及命名空间 std，同时做了一些 typedef；

2. 相机结构 Camera：由于使用的双目相机，那么除了常规的内参 fx, fy, cx, cy 以外，还需要记录基线 baseline，以及相机间的外参，同时在相机类里还增加了一些坐标系之

间的转换函数用作辅助（世界坐标系，相机坐标系以及图像坐标系）；

3. 可视化工具 Viewer：主要基于 OpenGL(Pangolin) 进行设置，基础画图和之前实现过的比较类似，主要是增加了更新功能，在插入一帧新的关键帧后增加对应特征点的以及关键帧处的相机位姿，同时还需要实时地将当前视野以及特征点显示出来；

4. g2o 类型 g2o_types：主要包括 Vertex 相关和 Edge 相关，Vertex 相关的又包括位姿和路标点，读写方面都差不多，主要是输入以及更新，分别如下所示：

   位姿 Vertex，更新量用于左乘当前估计值即可：

```
/// left multiplication
virtual void oplusImpl(const double *update) override {
    Eigen::Matrix<double, 6, 1> update_eigen;
    update_eigen << update[0], update[1], update[2],
            update[3], update[4], update[5];
    _estimate = Sophus::SE3d::exp(update_eigen) * _estimate;
}
```

   路标点 Vertex，更新可以直接加在估计值上:

```
/// left multiplication
virtual void oplusImpl(const double *update) override {
    _estimate[0] += update[0];
    _estimate[1] += update[1];
    _estimate[2] += update[2];
}
```

   Edge 类也有两种，一类用于前端计算位姿估计值，在这一步为了提高系统性能不对路标点进行更新，所以是一元边，在后端的 bundle adjustment 中，我们需要同时更新所有关键帧的路标点坐标以及相机位姿，所以是二元边，具体实现如下所示：

   位姿一元边，具体计算公式推导已在上一部分进行说明：

```
virtual void computeError() override {
    const VertexPose *v = static_cast<VertexPose *>(_vertices[0]);
    Sophus::SE3d T = v->estimate();
    Eigen::Vector3d pos_pixel = _K * (T * _pos3d);
    pos_pixel /= pos_pixel[2];
    _error = _measurement - pos_pixel.head<2>();
}
```

```
virtual void linearizeOplus() override {
    const VertexPose *v = static_cast<VertexPose *>(_vertices[0]);
    Sophus::SE3d T = v->estimate();
    Eigen::Vector3d pos_cam = T * _pos3d;
    double fx = _K(0, 0);
    double fy = _K(1, 1);
    double X = pos_cam[0];
    double Y = pos_cam[1];
    double Z = pos_cam[2];
    double Zinv = 1.0 / (Z + 1e-18);//to avoid overfloating
    double Zinv2 = Zinv * Zinv;
    double XY = X*Y;
    double X2 = X*X;
    double Y2 = Y*Y;
    Eigen::Matrix<double, 2, 6> dedxi;
    dedxi << -fx * Zinv, 0, fx * X * Zinv2,
            fx * XY * Zinv2, -fx - fx * X2 * Zinv2, fx * Y * Zinv,
            0, -fy * Zinv, fy * Y * Zinv2,
            fy + fy * Y2 * Zinv2, -fy * XY * Zinv2, -fy * X * Zinv;
    _jacobianOplusXi = dedxi;
}
```

位姿以及路标点二元边，大部分和一元边，除了增加了对路标点的雅克比矩阵，具体计算公式推导已在上一部分进行说明：

```
_jacobianOplusXj = dedxi.block<2, 3>(0, 0) *
                        _cam_ext.rotationMatrix() * T.rotationMatrix();
```

3.3 前端和后端

前后端工作流程主要如下图所示：

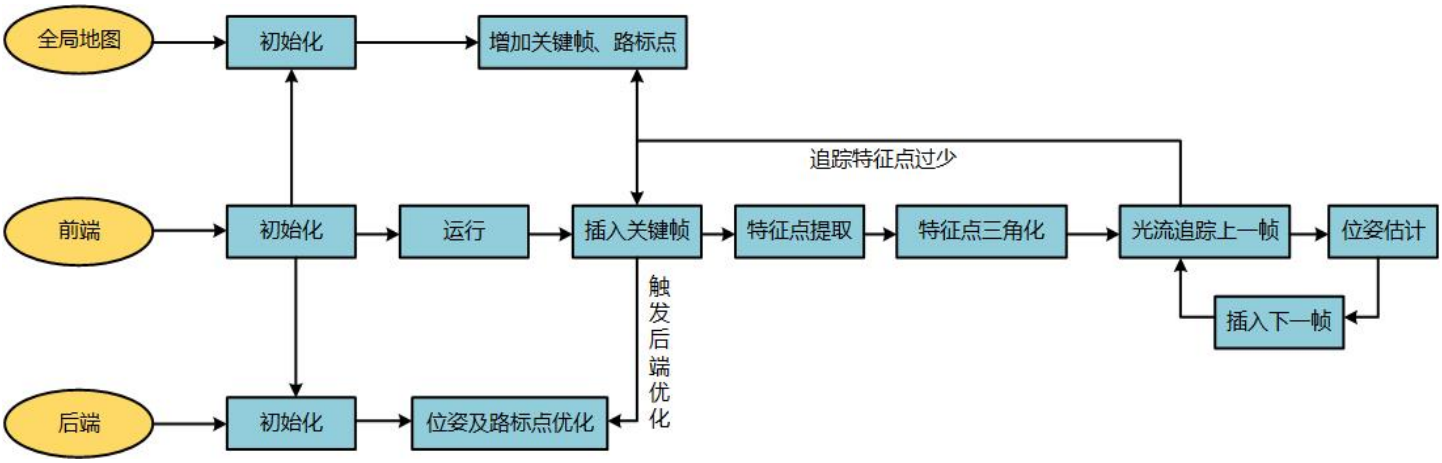


图 6: 系统工作流程

1. 前端工作流程主要包括：初始化、对上一帧进行光流追踪、提取关键帧（包括提取新的特征点，并建立路标殿、触发后端优化），下面详细说明：

   当前端初始化时，同时初始化对应的全局地图、后端以及可视化模块，然后进入运行阶段；运行阶段首先插入下一帧，判断是否是关键帧，判断标准是如果不存在上一帧，或者存在上一帧但上一帧光流追踪点至当前帧过少，则判断为关键帧，对关键帧进行特征点提取以及三角化路标点并放入可视化地图中；同时，每发现一个关键帧都会触发一次后端优化更新，这部分在后端再进行说明。以下列举部分代码进行说明：

   光流追踪上一帧，主要使用的是 Opencv 库中自带的函数 calcOpticalFlowPyrLK：

```
//calculate optical flow
//from left image of last frame to left image of current frame
cv::calcOpticalFlowPyrLK(
    last_frame_->left_img_, //上一帧左侧图像
    current_frame_->left_img_,  //当前帧左侧图像
    kps_last, //上一帧特征点（输入值）
    kps_current, //当前帧特征点（光流计算输出值）
    status, //每个特征点的追踪状态
    error, //每个特征点的追踪误差
    cv::Size(11, 11), //窗口大小
    3, //金字塔层数
    //判断准则
    cv::TermCriteria(cv::TermCriteria::COUNT+cv::TermCriteria::EPS, 30,0.01),
    cv::OPTFLOW_USE_INITIAL_FLOW);
```

三角化路标点，采用直接线性变换，具体原理推导在上一部分说明：

```
bool FrontEnd::triangulation(const vector<Sophus::SE3d> &poses,
                             const vector<Eigen::Vector3d> points,
                             Eigen::Vector3d &pWorlds) {
    Eigen::MatrixXd A(2 * poses.size(), 4);
    Eigen::VectorXd b(2 * poses.size());
    b.setZero();
    for (size_t i = 0; i < poses.size(); ++i) {
        Eigen::Matrix<double, 3, 4> m = poses[i].matrix3x4();
        A.block<1, 4>(2 * i, 0) = points[i][0] * m.row(2) - m.row(0);
        A.block<1, 4>(2 * i + 1, 0) = points[i][1] * m.row(2) - m.row(1);
    }
    auto svd = A.bdcSvd(Eigen::ComputeThinU | Eigen::ComputeThinV);
    pWorlds = (svd.matrixV().col(3) / svd.matrixV()(3, 3)).head<3>();

    if (svd.singularValues()[3] / svd.singularValues()[2] < 1e-2) {
        return true;
    }
    return false;
}
```

位姿估计的 g2o 实现部分和之前作业类似：

```
// setup g2o
typedef g2o::BlockSolver_6_3 BlockSolverType;
typedef g2o::LinearSolverDense<BlockSolverType::PoseMatrixType>
        LinearSolverType;
auto solver = new g2o::OptimizationAlgorithmLevenberg(
g2o::make_unique<BlockSolverType>(g2o::make_unique<LinearSolverType>()));
g2o::SparseOptimizer optimizer;
optimizer.setAlgorithm(solver);

// vertex
VertexPose *vertex_pose = new VertexPose();  // camera vertex_pose
vertex_pose->setId(0);
vertex_pose->setEstimate(current_frame_->Pose());
optimizer.addVertex(vertex_pose);

// K
Eigen::Matrix3d K = left_camera_->K();
```

```
// edges
int index = 1;
vector<EdgeProjectionPoseOnly *> edges;
vector<FeaturePtr> features;
for (size_t i = 0; i < current_frame_->left_features_.size(); ++i) {
    auto mp = current_frame_->left_features_[i]->map_point_.lock();
    if (mp) {
        features.push_back(current_frame_->left_features_[i]);
        EdgeProjectionPoseOnly *edge =
            new EdgeProjectionPoseOnly(mp->pos_, K);
        edge->setId(index);
        edge->setVertex(0, vertex_pose);
        cv::Point2f pt = current_frame_->left_features_[i]->position_.pt;
        Eigen::Vector2d pt_(pt.x, pt.y);
        edge->setMeasurement(pt_);
        edge->setInformation(Eigen::Matrix2d::Identity());
        edge->setRobustKernel(new g2o::RobustKernelHuber);
        edges.push_back(edge);
        optimizer.addEdge(edge);
        index++;
    }
}
```

2. 后端工作流程主要包括：接收前端传来的更新地图的信号，从 map 中获取关键帧和路标点通过 BA 进行更新，同时为了提高系统性能减少 BA 的规模，只对最近的若干个关键帧进行 BA 优化，关键帧保存数量是 7 张，后段部分主要涉及的只有 BA 优化，这部分和前端类似。

3.4 输入和输出

1. 数据输入

   首先在系统初始化时通过输入参数先初始化数据集目录位置，并初始化双目内参，该部分代码如下所示：

```
dataset_root_path_ = data_path; //数据集根目录
seq_id_ = seq_id; //对应序列
```

从 calib.txt 进行内参读入:

```
bool Visual_odometry::init_camera() {

    ifstream fin(dataset_root_path_ + seq_id_ + "/calib.txt");
    if (!fin) {
        cout << "cannot␣find␣" << dataset_root_path_
            << seq_id_ << "/calib.txt!" << endl;
        return false;
    }
    int idx = 0;
    while(idx < 2) {
        char camera_name[3];
        for (int k = 0; k < 3; ++k) {
            fin >> camera_name[k];
        }
        double projection_data[12];
        for (int k = 0; k < 12; ++k) {
            fin >> projection_data[k];
        }
        Eigen::Matrix3d K;
        Eigen::Vector3d t(0.0, 0.0, 0.0);
        K << projection_data[0], projection_data[1], projection_data[2],
            projection_data[4], projection_data[5], projection_data[6],
            projection_data[8], projection_data[9], projection_data[10];
        t << projection_data[3], projection_data[7], projection_data[11];
        t = K.inverse() * t;
        K = K * 0.5;
        if(idx == 0)
            cam1_ = shared_ptr<Camera>
            (new Camera(K(0,0), K(1,1), K(0,2), K(1,2),
                t.norm(), Sophus::SE3d(Sophus::SO3d(), t )));
        if(idx == 1)
            cam2_ = shared_ptr<Camera>
            (new Camera(K(0,0), K(1,1), K(0,2), K(1,2),
                t.norm(), Sophus::SE3d(Sophus::SO3d(), t )));
        idx++;
    }
}
```

图像输入采用 opencv 的图像读取函数，由于 kitti 原图过大还进行了一倍的缩小处理，具体函数如下所示：

```
cv::Mat left_image, right_image;
boost::format fmt("%s/image_%d/%06d.png");
left_image = cv::imread((fmt % (dataset_root_path_ + seq_id_) % 0
                    % current_index).str(), cv::IMREAD_GRAYSCALE);
right_image = cv::imread((fmt % (dataset_root_path_ + seq_id_) % 1
                    % current_index).str(), cv::IMREAD_GRAYSCALE);
if(!left_image.cols) return false;
cv::Mat image_left_resized, image_right_resized;
cv::resize(left_image, image_left_resized, cv::Size(),
            0.5, 0.5, cv::INTER_NEAREST);
cv::resize(right_image, image_right_resized, cv::Size(),
            0.5, 0.5, cv::INTER_NEAREST);
```

2. 结果输出和保存

由于搭建的是双目视觉里程计系统，所以不关心稀疏特征点地图的建立准确性，所以结果主要保存的是所有帧的位姿信息以及系统运行的平均时间，将其写入 result.txt 中，同时区分每个结果是否来自关键帧，代码如下所示：

```
ofstream outfile;
outfile.open(dataset_root_path_ + seq_id_ + "/result.txt");
for(int i = 0; i < allFrames_.size(); i++){
    if(allFrames_[i]->is_keyframe_){
        outfile << "key:␣";
    }
    else{
        outfile << "other:␣";
    }
    auto pose_ = allFrames_[i]->Pose();
    Eigen::Matrix<double,Eigen::Dynamic,Eigen::Dynamic, Eigen::RowMajor> pose_vec
    pose_vec.resize(1,12);
    outfile << pose_vec << "\n";

}
outfile.close();
```

4 实验结果

4.1 实验设置

4.2 程序运行

1. 由于 g2o 不同版本间差异较大,运行前先安装包里的 g2o;运行时,通过 cmake&&make 编译出可执行文件后,执行./vo [文件路径] [序列号]。如:./vo /data_odometry_gray/sequence/00

2. 结果可视化: 可以运行 scripts 文件夹中的 visualize_result.py, 脚本内容比较简单, 运行前先将路径改为运行结果 txt 文件以及 groundtruth 的 txt 文件路径。

4.3 双目 VO 结果

每次运行结果会存放在文件路径下的 result.txt，按 pose 格式存储。由于 cpp 不方便由于绘制曲线，所以使用 python 脚本对轨迹的三个方向分别进行显示，运行时的视频保存在附件中；

4.4 提取特征不同对系统性能的影响

为了测试不同角点的系统性能影响，选用 opencv 的 gftt 和 fast 角点检测进行比较，下面进行说明:

1. 使用 GFTT 角点提取时，关键帧提取较为密集，程序平均每帧处理时间如下图所示:

```
VO is running
VO cost time averaged: 0.016713 seconds for GFTT.
VO is running
```

图 7: GFTT 平均处理时间

2. 使用 FAST 特征提取时，关键帧提取相对少，程序平均每帧处理时间如下图所示:

```
VO is running
VO cost time averaged: 0.0170008 seconds for FAST.
VO is running
```

图 8: FAST 平均处理时间

可以发现两者单帧处理时间相近，下面进行精度的比较；

4.5 提取特征不同对系统精度的影响

总体而言，目前搭建的双目里程计系统均有不同程度的偏移，精度并不高。

1. 使用 GFTT 特征提取时，x 方向和 y 方向的结果如下图所示：

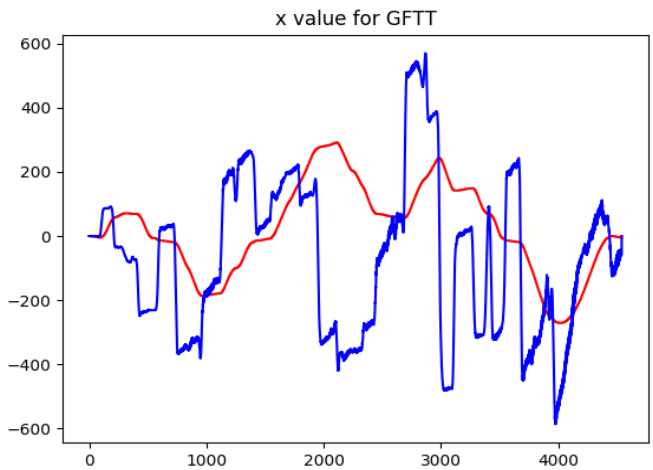


图 9: x 方向

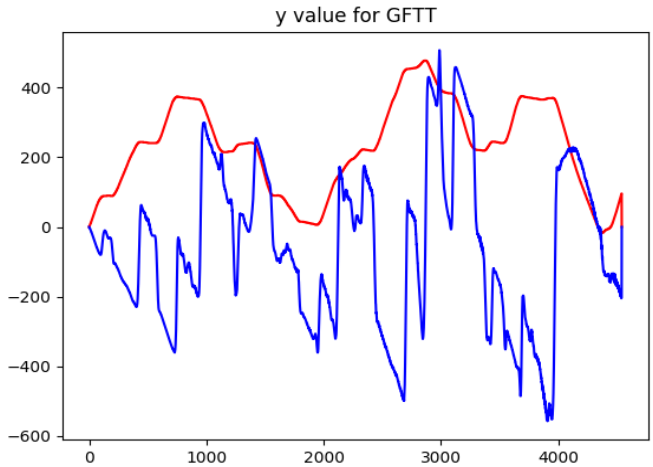


图 10: y 方向

2. 使用 FAST 特征提取时，x 方向和 y 方向的结果如下图所示：

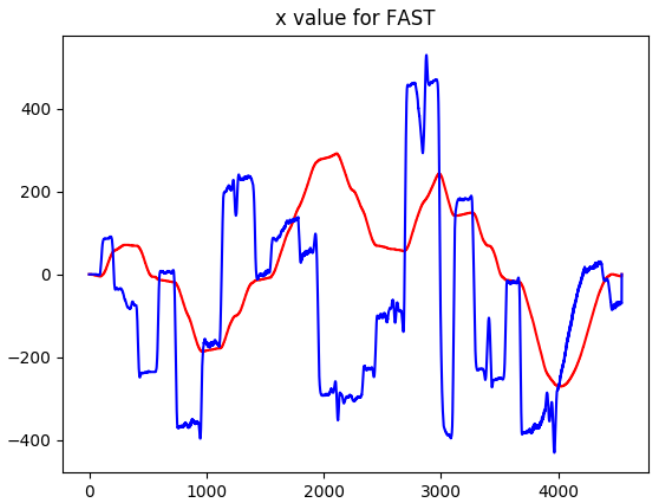


图 11: x 方向

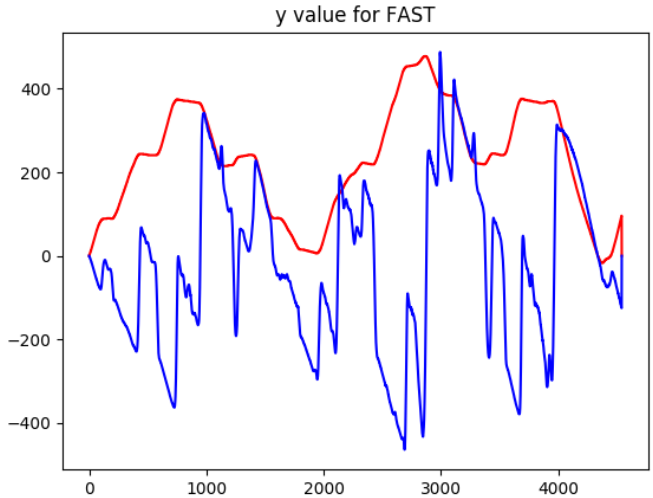


图 12: y 方向

总体来说，两个角点实现的效果精度都不是很好，具体原因在最后一部分分析，但是可以看出来相较而言 FAST 角点的表现略好一点。

4.6 结果分析及后续改进思路

可以看出目前实现的 VO 效果并不是很好，经过分析可能有以下原因：

1. 左右图特征点匹配方式不好，目前的做法是通过光流来匹配左右图的角点，这样误差可能会比较大，后续可以考虑针对不同角点采用不同描述子的方式来进行匹配；
2. 从系统运行性能而言采用光流也不是特别好的做法，由于 kitti 的数据图左右已经对齐，可以考虑将光流直接改成一维的搜索，即在同一行像素点中进行匹配选择，应该可以有效提高系统运行速度；
3. 目前系统在高度方向上的偏移过大，考虑可能是因为缺少回环检测模块，后续考虑增加回环检测应该能减少高度方向上的误差。

5 参考

1. https://blog.csdn.net/linolzhang/article/details/54342610
2. https://zhuanlan.zhihu.com/p/36382429
3. https://blog.csdn.net/jinzhichaoshuiping/article/details/93418989
4. https://www.cnblogs.com/yepeichu/p/10792899.html
5. https://math.stackexchange.com/questions/1087003/can-the-cross-product-be-a-matrix
6. http://paopaorobot.org/265.html
7. https://medium.com/@shehan.a.perera/a-comparison-of-sift-surf-and-orb-333d64bcaaea
8. https://cloud.tencent.com/developer/article/1367394
9. < 视觉 slam 十四讲 >
10. https://www.cnblogs.com/lsgxeva/p/7788061.html
11. https://blog.csdn.net/liangchunjiang/article/details/79869830
12. https://github.com/raulmur/ORB_SLAM2
13. https://github.com/gaoxiang12/slambook2